先进计算与数字工程研究中心

PATTERN RECOGNITION AND MACHINE LEARNING

Lecture 14 – Ensemble Learning II: Boosting, Gradient Boosting, and Stacking



Chizhi Chris ZHANG

zhangchizhi@ciomp.ac.cn

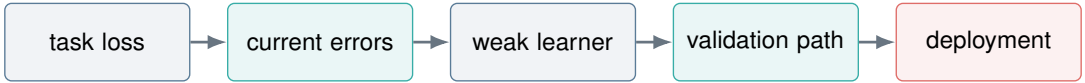
Advanced Computing and Digital Technology Research Center

University of Chinese Academy of Sciences

Fall 2026

先进计算与数字工程研究中心
ADVANCED COMPUTING AND DIGITAL TECHNOLOGY RESEARCH CENTER

Turn current error into the next learning signal



Sequential improvement is useful only while independent validation improves.

Guiding question

Should the next model reweight observations, follow a loss gradient, or combine out-of-fold predictions?

- AdaBoost changes sample attention
- gradient boosting follows loss-specific pseudo-residuals
- tree systems regularize split search
- stacking learns complementarity from out-of-fold predictions

From boosting equations to a leakage-safe ensemble

By the end

You can derive AdaBoost updates, interpret functional gradients, regularize boosted trees, distinguish XGBoost from LightGBM, and build an out-of-fold stack.

Period	Core content	Decision
1	AdaBoost and margins	which cases receive attention?
2	gradient boosting and tree systems	which correction and capacity?
3	regularization and stacking	when does extra complexity earn its cost?

Record the path, not only the final score

先进计算与数字工程研究中心
ADVANCED COMPUTING AND DIGITAL TECHNOLOGY RESEARCH CENTER

learning signal | validation boundary | system budget

What to calculate

Every experiment saves the loss, stage, validation boundary, calibration result, feature availability time, random seed, memory, and latency.

What information should the next weak learner receive from the current ensemble?



Every round converts current error into a new supervised target, then adds a controlled correction.

Question

What information should the next weak learner receive from the current ensemble?

Example

improving a shallow-tree credit-risk baseline

Main idea

Every round converts current error into a new supervised target, then adds a controlled correction.

Build an ensemble by adding one learner at a time

先进计算与数字工程研究中心
ADVANCED COMPUTING AND DIGITAL TECHNOLOGY RESEARCH CENTER

前向分步加法模型解释

■假设前$m-1$的学习器已经已知，则训练误差

$$f_m(\boldsymbol{x}) = \sum_{m=1}^{M} \alpha_m \phi_m(\boldsymbol{x})$$

$$E = \sum_{i=1}^{N} \exp\left(-y_i f_m(\boldsymbol{x}_i)\right)$$

$$= \sum_{i=1}^{N} \exp\left(-y_i f_{m-1}(\boldsymbol{x}_i) - y_i \alpha_m \phi_m(\boldsymbol{x}_i)\right)$$

$$= \sum_{i=1}^{N} w'_{m,i} \exp\left(-y_i \alpha_m \phi_m(\boldsymbol{x}_i)\right)$$

$$w'_{m,i} = \exp\left(-y_i f_{m-1}(\boldsymbol{x}_i)\right)$$

$$= \sum_{i=1}^{N} w'_{m,i} \exp(\alpha_m) \mathbb{I}\left(y_i \neq \phi_m(\mathbf{x}_i)\right) + \sum_{i=1}^{N} w'_{m,i} \exp(-\alpha_m) \mathbb{I}\left(y_i = \phi_1(\mathbf{x}_i)\right)$$

Sequential additive modeling: model and variables

Model and assumptions

Boosting builds an additive predictor by fitting each new weak learner to a loss-derived correction signal.

$$F_m(x) = F_{m-1}(x) + \eta \alpha_m h_m(x)$$

Symbol	Meaning	Status
F_m	ensemble after round m	current predictor
h_m	weak learner	new correction direction
$\eta\alpha_m$	step size	controls contribution

Sequential additive modeling: worked example

Quantity	Value	Meaning
current score	0.30	under-confident positive
weak correction	$+0.40$	new rule
$\eta = 0.25$	$+0.10$	updated score 0.40

What the result means

Keep base learners weak enough that the validation path reveals useful incremental corrections.

Sequential additive modeling: what to check

Main idea

Every round converts current error into a new supervised target, then adds a controlled correction.

What to check

round-wise training and validation loss

Limit

an unconstrained learner can consume the entire correction and make later rounds chase noise

Sequential additive modeling: implementation

1. initialize a constant predictor
2. compute the round-specific signal
3. fit a constrained weak learner
4. choose its step
5. record train and validation loss

Check the result

Check round-wise training and validation loss on training and held-out examples.

Common mistake

an unconstrained learner can consume the entire correction and make later rounds chase noise

Sequential additive modeling: common mistakes

Example

improving a shallow-tree credit-risk baseline

Explain the calculation and check round-wise training and validation loss, using a small example.

Common mistake

an unconstrained learner can consume the entire correction and make later rounds chase noise

What to do

Keep base learners weak enough that the validation path reveals useful incremental corrections.

Sequential additive modeling: check your understanding

Question

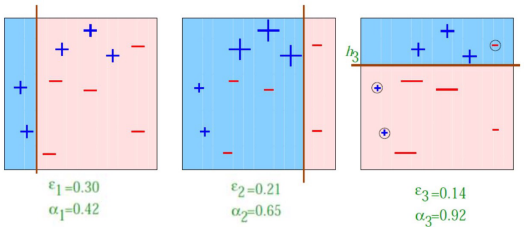
What information should the next weak learner receive from the current ensemble?

Answer

Every round converts current error into a new supervised target, then adds a controlled correction.

Decision. Keep base learners weak enough that the validation path reveals useful incremental corrections.

How does AdaBoost turn current classification mistakes into the next training distribution?



Question

How does AdaBoost turn current classification mistakes into the next training distribution?

Example

binary classification with a few suspected label errors

Main idea

Misclassified cases receive exponentially more attention, while correctly classified cases become relatively lighter.

AdaBoost increases weights on current mistakes

先进计算与数字工程研究中心
ADVANCED COMPUTING AND DIGITAL TECHNOLOGY RESEARCH CENTER

样本重新加权

- 如何加权？
  - 分对的样本，其权重除以d_1　　权重减小
  - 分错的样本，其权重乘以d_1　　权重增大
- d_1的值？
- 为了使得$\boldsymbol{w}_2$还是一个分布，$\boldsymbol{w}_1$乘以/除以d_1后的权重，再除以归一化因子：$Z_1 = \sum_{i=1}^{N} w_{1,i} d_1 \mathbb{I}(y_i \neq \phi_1(\mathbf{x}_i)) + \sum_{i=1}^{N} w_{1,i}/d_1 \, \mathbb{I}(y_i = \phi_1(\mathbf{x}_i))$，
- 即
  - 分对的样本的权重：$w_{2,i} = \frac{w_{1,i}/d_1}{Z_1}$
  - 分错的样本的权重：$w_{2,i} = \frac{w_{1,i} d_1}{Z_1}$

AdaBoost sample reweighting: model and variables

Model and assumptions

AdaBoost increases the normalized weight of misclassified observations and fits the next weak classifier on that weighted sample.

$$w_i^{(m+1)} = \frac{w_i^{(m)} \exp[-\alpha_m y_i h_m(x_i)]}{Z_m}, \quad \alpha_m = \frac{1}{2} \log \frac{1 - \epsilon_m}{\epsilon_m}$$

Symbol	Meaning	Status
$w_i^{(m)}$	sample weight	current attention
ϵ_m	weighted error	weak-rule quality
Z_m	normalizer	weights sum to one

AdaBoost sample reweighting: worked example

Quantity	Value	Meaning
ϵ_m	0.20	better than chance
α_m	0.693	positive vote
wrong case multiplier	$e^{0.693} = 2$	attention doubles

What the result means

Inspect high-weight cases and stop or robustify when attention concentrates on label noise.

The choice rule for AdaBoost sample reweighting

Main idea

Misclassified cases receive exponentially more attention, while correctly classified cases become relatively lighter.

What to check

weighted error and effective sample size by round

Limit

mislabeled observations can accumulate extreme weight and dominate later weak learners

AdaBoost sample reweighting: implementation

先进计算与数字工程研究中心
ADVANCED COMPUTING AND DIGITAL ENGINEERING RESEARCH CENTER

```
from sklearn.ensemble import AdaBoostClassifier
ada = AdaBoostClassifier(n_estimators=120, learning_rate=0.08, random_state=7)
ada.fit(Xc_train, yc_train)
print('AdaBoost train/test:', round(ada.score(Xc_train, yc_train), 3), round(ada.score(Xc_test, yc_test), 3))
```


Evidence and failure check

Check weighted error and effective sample size by round with the fitted model and data split; reject the run if mislabeled observations can accumulate extreme weight and dominate later weak learners.

AdaBoost sample reweighting: common mistakes

Example

binary classification with a few suspected label errors

Explain the calculation and check weighted error and effective sample size by round, using a small example.

Common mistake

mislabeled observations can accumulate extreme weight and dominate later weak learners

What to do

Inspect high-weight cases and stop or robustify when attention concentrates on label noise.

AdaBoost sample reweighting: check your understanding

Question

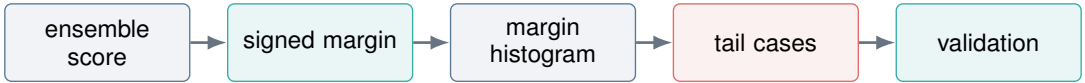
How does AdaBoost turn current classification mistakes into the next training distribution?

Answer

Misclassified cases receive exponentially more attention, while correctly classified cases become relatively lighter.

Decision. Inspect high-weight cases and stop or robustify when attention concentrates on label noise.

Why can training error reach zero while useful boosting progress continues?



Generalization is better diagnosed by the margin distribution than by zero-one training error alone.

Question

Why can training error reach zero while useful boosting progress continues?

Example

a classifier with zero training error after twenty rounds

Main idea

Generalization is better diagnosed by the margin distribution than by zero-one training error alone.

A margin combines the label and ensemble score

- For $y \in \{-1, +1\}$, the signed margin is $m = yF(x)$.
- $m > 0$: correct sign; $m < 0$: wrong sign.
- Exponential loss is $\exp(-m)$: margins 1, 0, and -1 give about 0.37, 1, and 2.72.
- Large negative margins dominate the loss and can make label noise influential.

Margins and exponential loss: model and variables

Model and assumptions

The normalized margin measures the signed ensemble confidence; AdaBoost continues increasing margins after labels become correct.

$$\rho_i = \frac{y_i \sum_m \alpha_m h_m(x_i)}{\sum_m |\alpha_m|}, \qquad L_i = \exp[-y_i F(x_i)]$$

Symbol	Meaning	Status
ρ_i	normalized margin	signed confidence
$F(x_i)$	ensemble score	pre-threshold output
L_i	exponential loss	emphasizes negative margins

Margins and exponential loss: worked example

Quantity	Value	Meaning
label	$y = +1$	positive case
score	$F(x) = 1.2$	correct
margin	$+1.2$	confident; low exponential loss

What the result means

Use validation margins together with task loss and calibration, not as a universal guarantee.

Margins and exponential loss: checking the result

Main idea

Generalization is better diagnosed by the margin distribution than by zero-one training error alone.

What to check

validation margin quantiles and exponential loss

Limit

maximizing margins on separable training data can still hurt probability calibration or minority cases

What to record when running Margins and exponential loss

1. save decision scores per round
2. multiply by labels
3. plot train and validation margins
4. inspect the lower tail
5. relate margin shifts to calibration

Check the result

Check validation margin quantiles and exponential loss on training and held-out examples.

Common mistake

maximizing margins on separable training data can still hurt probability calibration or minority cases

The warning sign for Margins and exponential loss

Example

a classifier with zero training error after twenty rounds

Explain the calculation and check validation margin quantiles and exponential loss, using a small example.

Common mistake

maximizing margins on separable training data can still hurt probability calibration or minority cases

What to do

Use validation margins together with task loss and calibration, not as a universal guarantee.

Explain Margins and exponential loss without naming the algorithm

Question

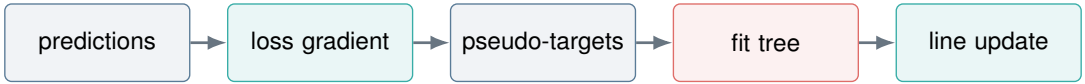
Why can training error reach zero while useful boosting progress continues?

Answer

Generalization is better diagnosed by the margin distribution than by zero-one training error alone.

Decision. Use validation margins together with task loss and calibration, not as a universal guarantee.

How can gradient descent optimize a function assembled from weak learners?



The pseudo-residual is a loss-specific direction in prediction space, not always an ordinary residual.

Question

How can gradient descent optimize a function assembled from weak learners?

Example

designing a boosting model for asymmetric prediction cost

Main idea

The pseudo-residual is a loss-specific direction in prediction space, not always an ordinary residual.

Gradient boosting follows the negative loss gradient

从Gradient Descent 到 Gradient Boosting

参数空间

$$\theta^t = \theta^{t-1} + \theta_t$$

第t次迭代后的参数　第t-1次迭代后的参数　第t次迭代的参数增量

$$\theta_t = -\eta_t g_t$$

参数更新方向为负梯度方向

$$\theta = \sum_{t=0}^{T} \theta_t$$

最终参数等于每次迭代的增量的累加和，

θ_0 为初值

函数空间

$$f^t(x) = f^{t-1}(x) + f_t(x)$$

第t次迭代后的函数　第t-1次迭代后的函数　第t次迭代的函数增量

$$f_t(x) = -\eta_t g_t(x)$$

同样地，拟合负梯度

$$F(x) = \sum_{t=0}^{T} f_t(x)$$

最终函数等于每次迭代的增量的累加和，

$f_0(x)$ 为模型初始值，通常为常数

From：http://wepon.me/files/gbdt.pdf

How Functional gradient descent changes the representation

Model and assumptions

Gradient boosting fits a base learner to the negative loss gradient evaluated at current predictions.

$$r_{im} = -\left.\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)}\right|_{F=F_{m-1}}, \quad h_m \approx r_m$$



Symbol	Meaning	Status
r_{im}	pseudo-residual	negative functional gradient
L	task loss	defines the correction
h_m	fitted direction	constrained approximation

Check the direction of change in Functional gradient descent

Quantity	Value	Meaning
prediction	0.6	current value
negative gradient	-0.4	desired correction
tree output	-0.3	approximated direction

What the result means

Derive the pseudo-residual from the chosen loss before selecting the learner or metric.

When Functional gradient descent earns a place in the pipeline

Main idea

The pseudo-residual is a loss-specific direction in prediction space, not always an ordinary residual.

What to check

loss decrease per stage on held-out data

Limit

treating every pseudo-residual as $y - F$ silently uses the squared-error loss for the wrong task

Functional gradient descent: implementation

1. define a differentiable loss
2. initialize the best constant
3. compute negative gradients
4. fit a shallow tree
5. choose leaf updates and step size

Check the result

Check loss decrease per stage on held-out data on training and held-out examples.

Common mistake

treating every pseudo-residual as $y - F$ silently uses the squared-error loss for the wrong task

When the objective of Functional gradient descent is not enough

Example

designing a boosting model for asymmetric prediction cost

Explain the calculation and check loss decrease per stage on held-out data, using a small example.

Common mistake

treating every pseudo-residual as $y - F$ silently uses the squared-error loss for the wrong task

What to do

Derive the pseudo-residual from the chosen loss before selecting the learner or metric.

A final decision on Functional gradient descent

Question

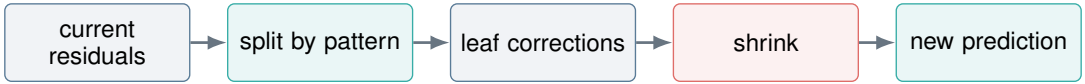
How can gradient descent optimize a function assembled from weak learners?

Answer

The pseudo-residual is a loss-specific direction in prediction space, not always an ordinary residual.

Decision. Derive the pseudo-residual from the chosen loss before selecting the learner or metric.

What does a regression tree contribute under squared-error boosting?



A tree does not predict the target anew; it explains structured error left by the current ensemble.

Question

What does a regression tree contribute under squared-error boosting?

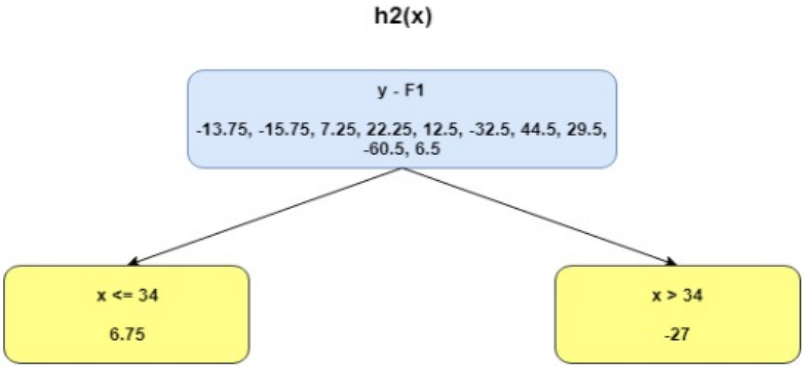
Example

predicting energy demand with nonlinear weather effects

Main idea

A tree does not predict the target anew; it explains structured error left by the current ensemble.

A regression tree fits the current residuals



This new tree fits residuals from the current ensemble, not the original target from scratch.

The variables that make Regression trees fit residual structure work

Model and assumptions

With squared error, pseudo-residuals equal ordinary residuals and each tree partitions input space into local corrections.

$$r_{im} = y_i - F_{m-1}(x_i), \qquad F_m(x) = F_{m-1}(x) + \eta \sum_{j=1}^{J_m} \gamma_{jm} \mathbf{1}[x \in R_{jm}]$$

Symbol	Meaning	Status
R_{jm}	leaf region	local error pattern
γ_{jm}	leaf correction	mean residual for squared loss
η	shrinkage	global step control

先进计算与数字技术研究中心
ADVANCED COMPUTING AND DIGITAL TECHNOLOGY RESEARCH CENTER

Read the numbers produced by Regression trees fit residual structure

Quantity	Value	Meaning
leaf residuals	$2, 1, -1$	three cases
leaf update	0.67	mean residual
$\eta = 0.1$	0.067	added correction

What the result means

Use shallow trees and inspect whether validation residual patterns actually disappear.

Three conditions for trusting Regression trees fit residual structure

Main idea

A tree does not predict the target anew; it explains structured error left by the current ensemble.

What to check

validation RMSE and residual structure

Limit

deep trees can fit isolated residual noise and produce a jagged function with poor extrapolation

An implementation checklist for Regression trees fit residual structure

```
from sklearn.ensemble import GradientBoostingRegressor
gbr = GradientBoostingRegressor(n_estimators=180, learning_rate=0.04, max_depth=2, subsample=0.8, random_state=7)
gbr.fit(Xr_train, yr_train)
print('GBR test R2:', round(gbr.score(Xr_test, yr_test), 3))
```

Evidence and failure check

Check validation RMSE and residual structure with the fitted model and data split; reject the run if deep trees can fit isolated residual noise and produce a jagged function with poor extrapolation.

Reject Regression trees fit residual structure under this condition

Example

predicting energy demand with nonlinear weather effects

Explain the calculation and check validation RMSE and residual structure, using a small example.

Common mistake

deep trees can fit isolated residual noise and produce a jagged function with poor extrapolation

What to do

Use shallow trees and inspect whether validation residual patterns actually disappear.

Regression trees fit residual structure: check your understanding

Question

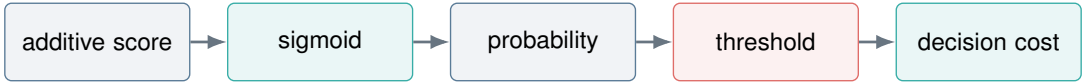
What does a regression tree contribute under squared-error boosting?

Answer

A tree does not predict the target anew; it explains structured error left by the current ensemble.

Decision. Use shallow trees and inspect whether validation residual patterns actually disappear.

Which score is boosted for classification, and how does it become a probability?



The additive score, probability, label, and decision threshold are four distinct objects.

Question

Which score is boosted for classification, and how does it become a probability?

Example

rare-event triage with a low intervention threshold

Main idea

The additive score, probability, label, and decision threshold are four distinct objects.

Classification boosting updates a probability score

logitBoosting

- 对K类分类问题，对标签$y = k$进行独热编码为向量： $\boldsymbol{y} = (0, 0, \ldots, 1, 0, \ldots, 0)^{\mathrm{T}}$，其中$y_k = 1$，$y_{k'} = 0$，$k' = 1, 2, \ldots, K$且$k' \neq K$
- 交叉熵损失：$L(\boldsymbol{f}(\boldsymbol{x}), \boldsymbol{y}) = -\sum_{k=1}^{K} y_k \ln(p_k)$，
- 其中

$$p_m(\boldsymbol{x}) = \frac{\exp\left(f_m(\boldsymbol{x})\right)}{\sum_{j=1}^{K} \exp\left(f_j(\boldsymbol{x})\right)} \qquad m = 1, 2, \ldots, K, \boldsymbol{f}(\boldsymbol{x}) = \left(f_1(\boldsymbol{x}), f_2(\boldsymbol{x}), \ldots, f_K(\boldsymbol{x})\right)$$

$$\frac{\partial L(\boldsymbol{f}(\boldsymbol{x}), y)}{\partial f_m} = \sum_{k=1}^{K} \frac{\partial L(\boldsymbol{f}(\boldsymbol{x}), y)}{\partial p_k} \frac{\partial p_k}{\partial f_m} = -\sum_{k=1}^{K} \frac{y_k}{p_k} \frac{\partial p_k}{\partial f_m} = p_m - y_m$$

Classification losses and probabilities: model and variables

Model and assumptions

Logistic gradient boosting updates a log-odds score whose sigmoid transform yields a class probability.

$$p(x) = \sigma(F(x)) = \frac{1}{1 + e^{-F(x)}}, \qquad r_i = y_i - p_i$$

Symbol	Meaning	Status
$F(x)$	logit score	additive model output
$p(x)$	estimated probability	sigmoid transform
r_i	log-loss pseudo-residual	probability error

Classification losses and probabilities: worked example

Quantity	Value	Meaning
score	1.386	log odds
probability	0.80	sigmoid result
threshold	0.35	cost-based positive decision

What the result means

Validate probabilities and set the decision threshold separately from the boosting fit.

A decision card for Classification losses and probabilities

Main idea

The additive score, probability, label, and decision threshold are four distinct objects.

What to check

log loss, calibration error, and threshold cost

Limit

using accuracy to tune the model and then reading raw scores as probabilities creates inconsistent decisions

Reproduce the result of Classification losses and probabilities

1. optimize log loss
2. save raw scores
3. convert to probabilities
4. evaluate calibration
5. choose threshold from operational cost

Check the result

Check log loss, calibration error, and threshold cost on training and held-out examples.

Common mistake

using accuracy to tune the model and then reading raw scores as probabilities creates inconsistent decisions

A stress test for Classification losses and probabilities

Example

rare-event triage with a low intervention threshold

Explain the calculation and check log loss, calibration error, and threshold cost, using a small example.

Common mistake

using accuracy to tune the model and then reading raw scores as probabilities creates inconsistent decisions

What to do

Validate probabilities and set the decision threshold separately from the boosting fit.

Classification losses and probabilities: check your understanding

Question

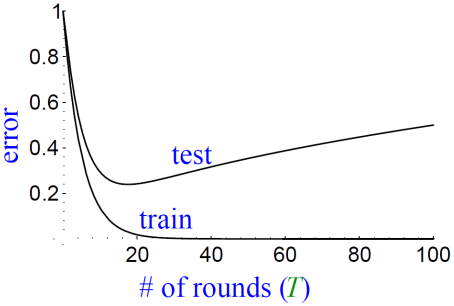
Which score is boosted for classification, and how does it become a probability?

Answer

The additive score, probability, label, and decision threshold are four distinct objects.

Decision. Validate probabilities and set the decision threshold separately from the boosting fit.

How do we control the path length and interaction complexity of a boosted model?



Question

How do we control the path length and interaction complexity of a boosted model?

Example

training a gradient-boosted model on noisy tabular data

Main idea

Learning rate and number of stages form a coupled budget; tree depth controls interaction order.

Shrinkage makes each tree contribute less

先进计算与数字工程研究中心
ADVANCED COMPUTING AND DIGITAL TECHNOLOGY RESEARCH CENTER

Shrinkage

- 通常对系数增加一个小的收缩因子(亦被称为学习率)，测试性能更好

$$f_m(\boldsymbol{x}_i) = f_{m-1}(\boldsymbol{x}_i) + \eta\phi_m(\boldsymbol{x}_i)$$

- 其中$0 < \eta < 1$ ，通常取一个较小的值，如$\eta = 0.01$。
- 较小的收缩因子通常意味着更多弱分学习器。

Shrinkage, depth, subsampling, and early stopping: model and variables

Model and assumptions

Boosting regularization combines small learning rates, constrained trees, stochastic row or feature sampling, and validation-based early stopping.

$$\mathcal{C} \approx \eta\, M\, J, \qquad M^{*} = \arg\min_{m} L_{\mathrm{valid}}(F_m)$$

Symbol	Meaning	Status
η	learning rate	step size
M	number of stages	path length
J	leaf or depth scale	interaction capacity

Shrinkage, depth, subsampling, and early stopping: worked example

Quantity	Value	Meaning
η	$0.1 o 0.03$	smaller steps
M	$100 o 350$	longer path
best round	247	restore this stage

What the result means

Use one validation protocol for early stopping and reserve an untouched test set for the final audit.

Shrinkage, depth, subsampling, and early stopping: checking the result

Main idea

Learning rate and number of stages form a coupled budget; tree depth controls interaction order.

What to check

best validation loss and stability of the selected stage

Limit

stopping on training loss or repeatedly inspecting the test set turns early stopping into leakage

Shrinkage, depth, subsampling, and early stopping: implementation

1. fix dependency-aware validation
2. start with shallow trees
3. search learning rate and stages jointly
4. monitor validation loss with patience
5. restore the best stage

Check the result

Check best validation loss and stability of the selected stage on training and held-out examples.

Common mistake

stopping on training loss or repeatedly inspecting the test set turns early stopping into leakage

What breaks first in Shrinkage, depth, subsampling, and early stopping

Example

training a gradient-boosted model on noisy tabular data

Explain the calculation and check best validation loss and stability of the selected stage, using a small example.

Common mistake

stopping on training loss or repeatedly inspecting the test set turns early stopping into leakage

What to do

Use one validation protocol for early stopping and reserve an untouched test set for the final audit.

Shrinkage, depth, subsampling, and early stopping: check your understanding

Question

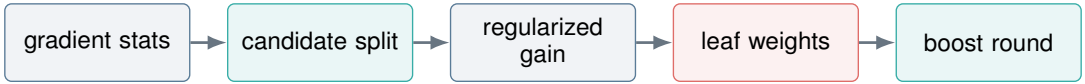
How do we control the path length and interaction complexity of a boosted model?

Answer

Learning rate and number of stages form a coupled budget; tree depth controls interaction order.

Decision. Use one validation protocol for early stopping and reserve an untouched test set for the final audit.

What does a second-order tree objective add beyond first-order gradient boosting?



Second-order curvature makes local updates explicit; regularization and missing-value routing belong to the learned tree structure.

Question

What does a second-order tree objective add beyond first-order gradient boosting?

Example

default-risk prediction with structured missing values

Main idea

Second-order curvature makes local updates explicit; regularization and missing-value routing belong to the learned tree structure.

XGBoost scores a split using gradients and curvature

先进计算与数字工程研究中心
ADVANCED COMPUTING AND DIGITAL TECHNOLOGY RESEARCH CENTER

分裂的增益度量

- ID3算法采用信息增益
- C4.5算法采用信息增益比
- CART采用Gini系数

- XGBoost： $J(f) = -\frac{1}{2}\sum_{t=1}^{T}\left[\frac{G_t^2}{H_t+\lambda}\right] + \gamma T$

XGBoost: regularized second-order trees: model and variables

Model and assumptions

XGBoost uses gradient and Hessian statistics to score candidate leaves while penalizing leaf count and weight magnitude.

$$\widetilde{\mathcal{L}}^{(m)} = \sum_i [g_i f_m(x_i) + \tfrac{1}{2} h_i f_m(x_i)^2] + \gamma T + \tfrac{1}{2}\lambda \sum_{j=1}^{T} w_j^2$$

Symbol	Meaning	Status
g_i, h_i	first and second derivatives	local loss geometry
T	number of leaves	structure cost
w_j	leaf weight	regularized update

XGBoost: regularized second-order trees: worked example

先进计算与数字工程研究中心
ADVANCED COMPUTING AND DIGITAL TECHNOLOGY RESEARCH CENTER

Quantity	Value	Meaning
G_j	-8	summed gradient
$H_j + \lambda$	10	regularized curvature
w_j^*	0.8	negative gradient ratio

What the result means

Audit feature availability and split stability before trusting a high-scoring boosted tree.

Accept or reject XGBoost: regularized second-order trees

Main idea

Second-order curvature makes local updates explicit; regularization and missing-value routing belong to the learned tree structure.

What to check

held-out loss, calibration, and split stability

Limit

aggressive tree search can exploit identifiers, missingness artifacts, or time leakage with impressive validation scores

XGBoost: regularized second-order trees: implementation

1. choose task objective
2. build fold-safe features
3. tune depth and minimum child weight
4. regularize leaves and subsample
5. early-stop with the deployment metric

Check the result

Check held-out loss, calibration, and split stability on training and held-out examples.

Common mistake

aggressive tree search can exploit identifiers, missingness artifacts, or time leakage with impressive validation scores

The hidden assumption in XGBoost: regularized second-order trees

Example

default-risk prediction with structured missing values

Explain the calculation and check held-out loss, calibration, and split stability, using a small example.

Common mistake

aggressive tree search can exploit identifiers, missingness artifacts, or time leakage with impressive validation scores

What to do

Audit feature availability and split stability before trusting a high-scoring boosted tree.

XGBoost: regularized second-order trees: check your understanding

Question

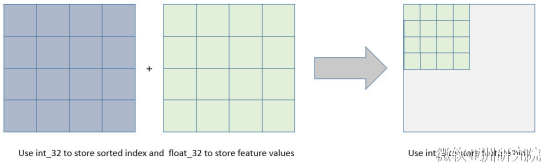
What does a second-order tree objective add beyond first-order gradient boosting?

Answer

Second-order curvature makes local updates explicit; regularization and missing-value routing belong to the learned tree structure.

Decision. Audit feature availability and split stability before trusting a high-scoring boosted tree.

How does LightGBM trade exact split search for speed and model capacity?



Question

How does LightGBM trade exact split search for speed and model capacity?

Example

a wide table with millions of rows and rare subgroups

Main idea

Histogram compression accelerates statistics; leaf-wise growth can create deep local paths and therefore needs explicit constraints.

先进计算与数字工程研究中心
ADVANCED COMPUTING AND DIGITAL TECHNOLOGY RESEARCH CENTER

Leaf-wise growth splits the most useful leaf

带有深度限制的按叶子生长 (leaf-wise) 算法

- Level-wise过一次数据可以同时分裂同一层的叶子，容易进行多线程优化，不容易过拟合
- 但Level-wise不加区分的对待同一层的叶子，带来了很多没必要的开销，比较低效（分裂增益较低，没必要进行搜索和分裂）
- Leaf-wise每次从当前所有叶子中，找到分裂增益最大(一般也是数据量最大)的一个叶子进行分裂。同 level-wise 相比，在分裂次数相同的情况下，leaf-wise 可以降低更多的误差，得到更好的精度。
- Leaf-wise 的缺点是可能会长出比较深的决策树，产生过拟合。因此 LightGBM 在leaf-wise 之上增加了一个最大深度的限制，在保证高效率的同时防止过拟合。

LightGBM: histograms and leaf-wise growth: model and variables

Model and assumptions

LightGBM bins continuous values into histograms and commonly grows the leaf with the largest loss reduction.

$$G_b = \sum_{i:\,\mathrm{bin}(x_{ij})=b} g_i, \qquad H_b = \sum_{i:\,\mathrm{bin}(x_{ij})=b} h_i$$

Symbol	Meaning	Status
b	feature bin	compressed split candidate
G_b, H_b	binned derivative sums	histogram statistics
leaf-wise	best-leaf expansion	capacity concentrates locally

LightGBM: histograms and leaf-wise growth: worked example

Quantity	Value	Meaning
raw values	10^6	large table
bins	255	bounded statistics
max leaves	31	explicit capacity cap

What the result means

Constrain leaves and minimum leaf data before increasing histogram or iteration budgets.

Separate a good fit from a good decision in LightGBM: histograms and leaf-wise growth

Main idea

Histogram compression accelerates statistics; leaf-wise growth can create deep local paths and therefore needs explicit constraints.

What to check

validation loss, maximum path depth, memory, and latency

Limit

leaf-wise growth on small data can create a very deep branch that memorizes a narrow subgroup

LightGBM: histograms and leaf-wise growth: implementation

1. define categorical and missing-value handling
2. set leaf count and minimum leaf data
3. cap depth when data are small
4. subsample rows and features
5. profile memory and latency

Check the result

Check validation loss, maximum path depth, memory, and latency on training and held-out examples.

Common mistake

leaf-wise growth on small data can create a very deep branch that memorizes a narrow subgroup

Do not trust LightGBM: histograms and leaf-wise growth before this check

Example

a wide table with millions of rows and rare subgroups

Explain the calculation and check validation loss, maximum path depth, memory, and latency, using a small example.

Common mistake

leaf-wise growth on small data can create a very deep branch that memorizes a narrow subgroup

What to do

Constrain leaves and minimum leaf data before increasing histogram or iteration budgets.

LightGBM: histograms and leaf-wise growth: check your understanding

Question

How does LightGBM trade exact split search for speed and model capacity?

Answer

Histogram compression accelerates statistics; leaf-wise growth can create deep local paths and therefore needs explicit constraints.

Decision. Constrain leaves and minimum leaf data before increasing histogram or iteration budgets.

How can a meta-learner combine models without seeing their in-sample predictions?



Each meta-training prediction must come from a base model that did not train on that observation.

Question

How can a meta-learner combine models without seeing their in-sample predictions?

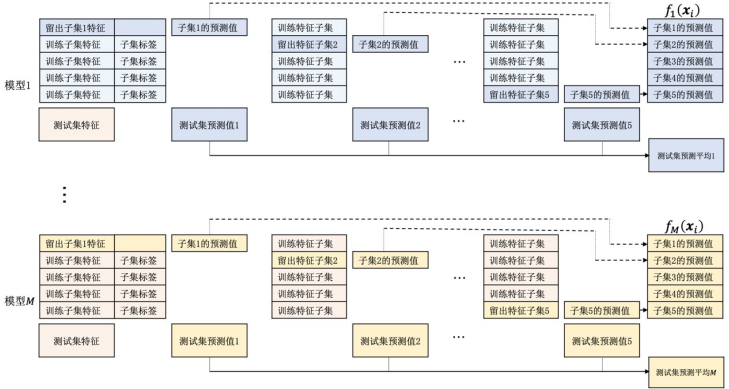
Example

combining linear, forest, and boosted models for tabular classification

Main idea

Each meta-training prediction must come from a base model that did not train on that observation.

Stacking learns from out-of-fold predictions



Each training row receives base-model predictions from folds that did not train on that row.

Out-of-fold stacking and deployment: model and variables

Model and assumptions

Stacking trains a meta-model on out-of-fold base predictions, then refits base models and recreates the same feature contract for inference.

$$z_{ik} = f_k^{(-\mathrm{fold}(i))}(x_i), \qquad \widehat{y}_i = g(z_{i1}, \ldots, z_{iK})$$

Symbol	Meaning	Status
z_{ik}	out-of-fold prediction	leakage-safe meta-feature
f_k	base model	diverse predictor
g	meta-learner	combines predictions

Out-of-fold stacking and deployment: worked example

Quantity	Value	Meaning
five folds	80/20	base train/OOF
three models	3 scores	meta-feature row
meta-model	logistic	stable combination

What the result means

Keep stacking only when outer validation shows stable incremental gain after cost and calibration checks.

Out-of-fold stacking and deployment: checking the result

Main idea

Each meta-training prediction must come from a base model that did not train on that observation.

What to check

outer-fold gain over the best base model

Limit

training the meta-model on in-sample base predictions produces an unrealistically powerful and non-reproducible stack

Out-of-fold stacking and deployment: implementation

```
from sklearn.ensemble import RandomForestClassifier, StackingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
base = [('linear', make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000))), ('forest',
     RandomForestClassifier(n_estimators=120, min_samples_leaf=3, random_state=7))]
stack = StackingClassifier(estimators=base, final_estimator=LogisticRegression(), cv=5, stack_method='predict_proba
     ')
stack.fit(Xc_train, yc_train)
print('Stack test:', round(stack.score(Xc_test, yc_test), 3))
```

Evidence and failure check

Check outer-fold gain over the best base model with the fitted model and data split; reject the run if training the meta-model on in-sample base predictions produces an unrealistically powerful and non-reproducible stack.

Out-of-fold stacking and deployment: limitations

ADVANCED COMPUTING AND DIGITAL TECHNOLOGY RESEARCH CENTER

Example

combining linear, forest, and boosted models for tabular classification

Explain the calculation and check outer-fold gain over the best base model, using a small example.

Common mistake

training the meta-model on in-sample base predictions produces an unrealistically powerful and non-reproducible stack

What to do

Keep stacking only when outer validation shows stable incremental gain after cost and calibration checks.

Out-of-fold stacking and deployment: check your understanding

Question

How can a meta-learner combine models without seeing their in-sample predictions?

Answer

Each meta-training prediction must come from a base model that did not train on that observation.

Decision. Keep stacking only when outer validation shows stable incremental gain after cost and calibration checks.

先进计算与数字工程研究中心
ADVANCED COMPUTING AND DIGITAL TECHNOLOGY RESEARCH CENTER

A method matrix for the final ensemble decision

Decision

Use AdaBoost for reweighting simple classifiers, GBDT for additive loss minimization, regularized tree boosting for tabular systems, and stacking for complementary model families.

Evidence

Compare them under the same split, metric, calibration check, and serving budget.

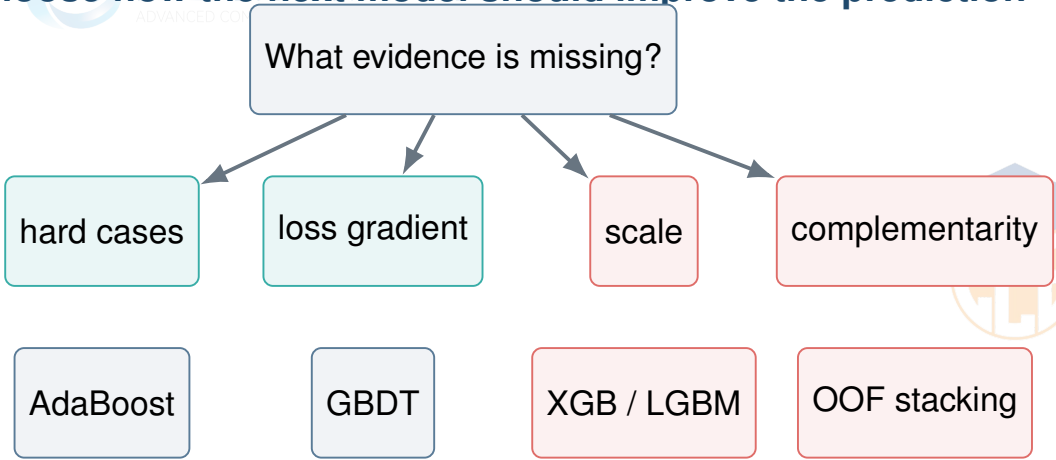
Stop condition

Reject any stacked result whose meta-learner has seen in-fold base predictions.

Compare ensemble mechanisms by their training signal

Method	Next learner sees	Primary control
AdaBoost	weighted classification errors	weak-rule error and margins
GBDT	negative loss gradients	learning rate, depth, early stop
XGBoost / LightGBM	derivative histograms	regularized structure and systems limits
stacking	out-of-fold base predictions	outer validation and simple meta-model

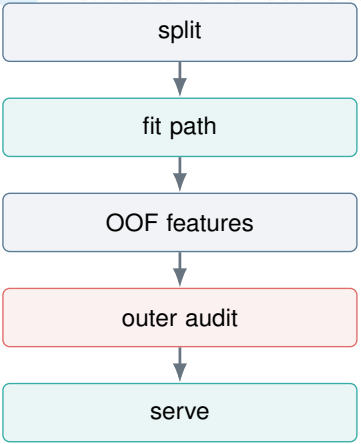
Choose how the next model should improve the prediction



Start from the deliverable

Identify whether the baseline needs harder-case attention, loss-specific residual correction, scalable tree search, or stable model complementarity.

A leakage-safe ensemble workflow



1. freeze the independent sampling unit and time boundary
2. fit all preprocessing inside development folds
3. track stage-wise validation loss and calibration
4. generate only out-of-fold meta-features
5. audit subgroups, missingness, and feature availability
6. profile the complete inference graph

The test set is evidence, not a steering signal.

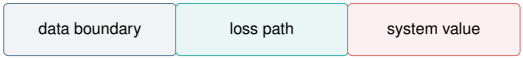
Quantify incremental value after cost

$$\Delta U = U(\text{ensemble}) - \max_k U(f_k) - \lambda_{\text{lat}} \Delta t - \lambda_{\text{mem}} \Delta m$$

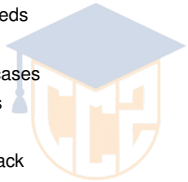
Observation	Likely issue	Next action
train improves; validation worsens	path too long	early-stop or shrink
stack wins in-sample only	meta leakage	rebuild OOF features
tiny score gain; large latency	poor system value	prefer simpler model

Stress-test before deployment

ADVANCED COMPUTING AND DIGITAL TECHNOLOGY RESEARCH CENTER



- repeat dependency-aware folds and seeds
- plot loss and margins by stage
- inspect high-weight and high-gradient cases
- audit missing-value and time semantics
- recalibrate on held-out predictions
- measure tail latency of the complete stack



Six takeaways to retain

先进计算与数字工程研究中心
ADVANCED COMPUTING AND DIGITAL TECHNOLOGY RESEARCH CENTER

1. boosting is staged loss correction
2. AdaBoost reweights classification errors
3. GBDT fits loss-specific negative gradients
4. learning rate and stages are coupled
5. tree systems differ in optimization and growth strategy
6. stacking requires out-of-fold meta-features

Exit ticket: defend one ensemble design

Prompt

Choose AdaBoost, GBDT, XGBoost/LightGBM, or stacking. State the learning signal, one capacity control, one leakage check, and one deployment metric.

Case

A boosted-tree baseline improves AUROC slightly after 300 rounds, but log loss worsens after round 170 and a three-model stack adds 12 ms latency.